PRECISION AUTOMATION FRAMEWORK

UI Module — Technical Documentation

Prepared By	**P. Raghu Vamsi**
Role	UI Module Lead
Module	`ui-module/`
Application Under Test	AutomationExercise.com
Tech Stack	Java 17, Selenium WebDriver 4, Cucumber, TestNG, Apache POI, ExtentReports

1. Module Overview

The ui-module is the browser-based test automation module of the Precision Automation Framework. It is responsible for end-to-end validation of the AutomationExercise e-commerce web application through a browser using Selenium WebDriver 4.

The module follows the Page Object Model (POM) design pattern, where each web page of the application is represented as a dedicated Java class. This ensures a clean separation between test logic (step definitions) and page interaction logic (page classes), making the tests maintainable and readable.

All test scenarios are written in BDD Gherkin syntax using Cucumber feature files. Test data is loaded from an Excel workbook (testdata.xlsx) via Apache POI, making the tests fully data-driven with zero hardcoded values in test code.

1.1 Objectives

- Automate critical user journeys on the AutomationExercise e-commerce website
- Validate user registration, login (valid and invalid), and cart operations
- Follow Page Object Model for clean separation of page logic from test logic
- Drive all test data from Excel sheets using Apache POI
- Generate detailed HTML execution reports via ExtentReports with failure screenshots
- Support multi-browser execution — Chrome, Firefox, and Edge

1.2 Test Coverage Summary

#	Test Case	Description	Page(s) Involved	Data Source
TC1	User Registration	Full registration form with title, DOB, address, country, etc.	RegisterPage	Excel - RegisterData
TC2	Valid Login	Login with correct credentials, verify 'Logged in as' visible	LoginPage, HomePage	Excel - LoginData
TC3	Invalid Login	Login with wrong credentials, verify error message displayed	LoginPage	Excel - LoginData
TC4	Add Product to Cart	Navigate to products, add item, verify in cart with quantity	ProductPage, CartPage	Excel - CartData
TC5	Remove Product from Cart	Remove item from cart, verify cart becomes empty	CartPage	Excel - CartData

2. Module Architecture & Package Structure

The ui-module is organized into clean sub-packages under com.precision.ui. Each package has a distinct responsibility — pages for UI interactions, stepdefs for Cucumber glue, hooks for lifecycle management, and runners for test execution.

2.1 Package Structure

```
ui-module/
  src/
    main/java/com/precision/ui/
      pages/           BasePage.java
                       HomePage.java
                       LoginPage.java
                       RegisterPage.java
                       ProductPage.java
                       CartPage.java
    test/java/com/precision/ui/
      stepdefs/        CommonSteps.java
                       LoginSteps.java
                       RegisterSteps.java
                       CartSteps.java
      hooks/           Hooks.java
      runners/         LoginTestRunner.java
                       RegisterTestRunner.java
                       CartTestRunner.java
    test/resources/features/
                       login.feature
                       register.feature
                       cart.feature
```

2.2 Layer Responsibilities

Layer	Class(es)	Responsibility
Base Page	BasePage	Abstract base with navigation, waits, JS utilities, ad-dismissal, and screenshot. All page classes extend this.
Pages	HomePage, LoginPage, RegisterPage, ProductPage, CartPage	Each class represents one web page. Contains @FindBy locators and user-action methods built on BasePage utilities.
StepDefs	CommonSteps, LoginSteps, RegisterSteps, CartSteps	Cucumber Gherkin step implementations. Wire feature file steps to page object methods. Use constructor injection to share state.

Hooks	Hooks	@Before initializes WebDriver and ExtentTest. @After captures screenshot on failure, marks pass/fail, quits driver.
Runners	LoginTestRunner, RegisterTestRunner, CartTestRunner	TestNG + Cucumber runners. Each runner maps to one feature file with specific tags and report paths.

2.3 Design Pattern — Page Object Model (POM)

POM is the core design pattern of the UI module. Each web page of the application under test is represented by a dedicated Java class. The page class encapsulates:

- All WebElement locators declared using @FindBy annotations with CSS selectors or XPath
- User-action methods (click, type, select, scroll) that interact with those elements
- State-check methods (isLoaded, isUserLoggedIn, isCartEmpty) that return booleans for assertions
- Fluent method chaining where actions return the next expected page object (e.g., clickLoginButton() returns new HomePage())

3. Class-by-Class Technical Documentation

3.1 BasePage.java

Package: com.precision.ui.pages

BasePage is the abstract foundation that all page classes extend. It initializes the WebDriver and Logger in its constructor and provides a comprehensive set of reusable utility methods so page classes never interact with Selenium APIs directly.

Constructor

- Retrieves the ThreadLocal WebDriver instance from DriverManager.getDriver()
- Initializes Log4j2 Logger scoped to the subclass using LogManager.getLogger(this.getClass())
- Calls PageFactory.initElements(driver, this) to inject all @FindBy annotated WebElements

Navigation Methods

Method	Description
navigateToBaseUrl()	Reads base.url from config.properties and navigates the browser to it
navigateTo(path)	Appends the given path to the base URL and navigates. Used by HomePage to go to /products
getPageTitle()	Returns the current browser window title string
getCurrentUrl()	Returns the current browser URL string
refreshPage()	Triggers browser refresh via driver.navigate().refresh()
goBack()	Triggers browser back navigation
waitForPageLoad()	Waits up to 15 seconds for document.readyState to equal 'complete' using JavascriptExecutor
waitForUrlToChange(part)	Waits until the current URL no longer contains the given string — used after login/registration redirects

Element Action Methods

Method	Description
click(element)	Waits for element to be clickable (via WaitUtils), then clicks

clearAndType(element, text)	Waits for visibility, clears existing text, types new text. Used for all input fields
getText(element)	Waits for visibility and returns the visible text of the element
scrollToElement(element)	Uses JavascriptExecutor to scroll the element into the visible viewport
jsClick(element)	Performs a JavaScript-based click — used when normal click is intercepted by overlays or ads
dismissAdsIfPresent()	Injects JS to hide all known ad selectors (iframes, divs, ins tags) by setting display:none. Called before critical interactions
takeScreenshot(name)	Captures a PNG screenshot and saves it to screenshots/ directory with the given name

3.2 HomePage.java

Package: com.precision.ui.pages

HomePage represents the landing page of the AutomationExercise application. It is the entry point of nearly all test scenarios and provides navigation to the Login page, Cart page, and Products page.

Element / Method	Locator	Purpose
homeCarousel	id='slider-carousel'	Used in isHomePageLoaded() to confirm page has fully rendered
signupLoginBtn	a[href='/login']	Clicked to navigate to the Login/Register page
cartBtn	a[href='/view_cart']	Clicked to navigate to the Cart page
logoutBtn	a[href='/logout']	Clicked to log the current user out
loggedInUsername	//a[contains(.,'Logged in as')]/b	Used in isUserLoggedIn() and getLoggedInUsername() to verify login state
navigateToProductsPage()	—	Calls navigateTo('/products'), waits for page load, dismisses ads, returns new ProductPage()

3.3 LoginPage.java

Package: com.precision.ui.pages

LoginPage handles both valid and invalid login flows. It implements a robust click fallback chain to handle real-world browser interferences such as ad overlays and element interception.

Element / Method	Locator	Purpose

loginHeading	//h2[normalize-space()='Login to your account']	Used in isLoginPageLoaded() to verify correct page
emailField	input[data-qa='login-email']	Email input field
passwordField	input[data-qa='login-password']	Password input field
loginButton	//button[normalize-space()='Login']	Submit button — uses 3-level fallback click strategy
errorMessage	//p[normalize-space()='Your email or password is incorrect!']	Error text verified in negative login scenario
loginAs(email, password)	—	Dismisses ads, types credentials, attempts click via normal → JS → Actions fallback chain, returns new HomePage()
clickLoginButtonExpectingFailure()	—	Clicks login without returning HomePage — used for invalid login where redirect should NOT happen

Click Fallback Chain

The loginAs() method implements a 3-level click fallback strategy to handle browser-level ad interception:

1. waitForClickability(loginButton).click() — standard Selenium click with explicit wait
2. jsClick(loginButton) — JavaScript-based click bypassing DOM event interception
3. Actions.moveToElement(loginButton).click().perform() — mouse simulation via Actions API

3.4 RegisterPage.java

Package: com.precision.ui.pages

RegisterPage is the most complex page class in the ui-module, handling the multi-step registration form that includes personal details, date of birth dropdowns, address fields, and country selection.

Signup Section Locators

Field	Locator	Method
Signup Name	input[data-qa='signup-name']	enterSignupName(name)
Signup Email	input[data-qa='signup-email']	enterSignupEmail(email)
Signup Button	button[data-qa='signup-button']	clickSignupButton()

Registration Form Locators

Field	Locator Type	Locator Value	Method
Title (Mr/Mrs)	id	id_gender1 / id_gender2	selectTitle(title)
Password	id	password	enterPassword(pwd)
DOB Day	id	days	selectDobDay(day)
DOB Month	id	months	selectDobMonth(month)
DOB Year	id	years	selectDobYear(year)
First Name	id	first_name	enterFirstName(fn)
Last Name	id	last_name	enterLastName(ln)
Address	id	address1	enterAddress(addr)
Country	id	country	selectCountry(country)
State	id	state	enterState(state)
City	id	city	enterCity(city)
Zipcode	id	zipcode	enterZipcode(zip)
Mobile	id	mobile_number	enterMobile(mobile)

Key Methods

- fillRegistrationForm(Map<String,String> data) — Orchestrator method that calls all individual field setters in sequence using data loaded from Excel. Called by RegisterSteps.
- clickCreateAccount() — Dismisses ads, scrolls to button, waits 1 second, JS-clicks the button, waits for redirect. Handles ad overlay interception.
- isAccountCreated() — Waits for the 'Account Created!' heading and returns true if visible.
- clickContinue() — Clicks the continue button after successful registration and returns new HomePage().

3.5 ProductPage.java

Package: com.precision.ui.pages

ProductPage represents the products listing page. It handles product card interactions, add-to-cart modal management, and navigation to the cart.

Element	Locator	Purpose
productNames	.productinfo.text-center p	List of product name text elements for getName(index)
productCards	.product-image-wrapper	List of product card wrappers used for scroll and count

addToCartButtons	a[data-product-id]	List of 'Add to Cart' buttons, one per product card
modalTitle	#cartModal .modal-title	Confirmation modal title — waited on after adding to cart
continueShoppingBtn	button[data-dismiss='modal']	Closes the cart modal to continue browsing
viewCartBtn	a[href='/view_cart']	Navigates to cart page, returns new CartPage()

addProductToCart(index)

This method dismisses ads first, then scrolls to the product card at the given index, performs a JS click on the add-to-cart button, and waits for the confirmation modal to appear. This sequence handles the common issue of ads blocking the add-to-cart button.

3.6 CartPage.java

Package: com.precision.ui.pages

CartPage represents the shopping cart page. It provides methods to inspect cart contents, retrieve item prices and quantities, remove items, and verify empty cart state.

Method	Returns	Description
isCartPageLoaded()	boolean	Waits for cart heading 'Shopping Cart' and returns isDisplayed()
getCartItemCount()	int	Returns the number of rows in the cart items table body
isCartEmpty()	boolean	Checks for 'Cart is empty!' message element. Returns false if element not found (no exception thrown)
removeItemFromCart(i)	void	Clicks delete button at given index, then waits for that cart row to become invisible
getItemPrice(index)	String	Returns the price text of the item at the given index
getItemQuantity(index)	String	Returns the quantity button text of the item at the given index

4. Step Definitions & Cucumber Glue

Step definition classes translate Gherkin steps into Java method calls on page objects. Cucumber's PicoContainer dependency injection is used to share state between step classes via constructor injection.

4.1 CommonSteps.java

CommonSteps holds the shared page references (homePage, loginPage) that other step classes depend on. It implements the steps common to all three feature files.

Gherkin Step	Action
user launches the application	Creates new HomePage(), calls navigateToBaseUrl()
user clicks on Signup Login button	Calls homePage.navigateToLoginPage(), stores result in loginPage
user should be logged in successfully	Asserts homePage.isUserLoggedIn() is true, logs the username

4.2 LoginSteps.java

LoginSteps handles both valid and invalid login step implementations. It uses constructor injection to receive the shared CommonSteps instance.

Gherkin Step	Action
user logs in with valid credentials	Reads TC2_ValidLogin row from Excel, prints debug info, calls loginPage.loginAs(email, password)
user logs in with invalid credentials	Reads TC3_InvalidLogin row from Excel, calls enterEmail(), enterPassword(), clickLoginButtonExpectingFailure()
error message should be displayed	Asserts loginPage.isErrorMessageDisplayed() is true

4.3 RegisterSteps.java

RegisterSteps handles the full registration flow. It generates a unique email per run to prevent conflicts with already-registered accounts.

Gherkin Step	Action
user enters signup name and email	Reads TC1_RegisterUser from Excel, generates unique email with timestamp, calls enterSignupName() and enterSignupEmail()

user clicks signup button	Calls registerPage.clickSignupButton()
user fills the registration form	Calls registerPage.fillRegistrationForm(registerData) — passes full Excel data map
user clicks create account button	Calls registerPage.clickCreateAccount(), prints current URL for debugging
account created message should be displayed	Asserts registerPage.isAccountCreated() is true
user clicks continue button	Calls registerPage.clickContinue(), stores returned HomePage in common.homePage

4.4 CartSteps.java

CartSteps manages the add-to-cart and remove-from-cart flows. It reads product index and expected values from Excel (CartData sheet) and verifies the cart state after each action.

Gherkin Step	Action
user adds first product to cart	Reads TC4_AddToCart from Excel, gets productIndex, navigates to products page, dismisses ads, adds product, stores product name
user clicks continue shopping	Calls productPage.clickContinueShopping() to close the cart modal
user clicks on Cart button	Calls homePage.navigateToCartPage(), stores CartPage reference
cart page should be displayed	Asserts cartPage.isCartPageLoaded() is true
cart should contain the added product	Asserts cart item count > 0, asserts quantity at index 0 matches Excel expected value
user removes item at index 0	Calls cartPage.removeItemFromCart(0)
cart should be empty	Asserts cartPage.isCartEmpty() is true

5. Hooks & Test Runners

5.1 Hooks.java

Package: com.precision.ui.hooks

Hooks manages the complete WebDriver and reporting lifecycle for every scenario. It is shared across all three runners via the glue path com.precision.ui.hooks.

Hook	Actions Performed
@Before	1. Calls ExtentTestManager.createTest(scenario.getName()) to register scenario in report. 2. Logs scenario start. 3. Calls DriverManager.initDriver() to launch the configured browser.
@After	1. If scenario failed: captures Base64 screenshot from TakesScreenshot, attaches to ExtentReport via MediaEntityBuilder, marks scenario as failed. 2. If scenario passed: marks as passed. 3. Always: quits driver, removes ExtentTest, flushes report.

5.2 Test Runners

Three separate TestNG Cucumber runners are provided — one per feature file. This allows targeted execution of individual test suites via tags.

Runner Class	Feature File	Tag	Report Path
LoginTestRunner	login.feature	@login	target/cucumber-reports/login-report.html
RegisterTestRunner	register.feature	@register	target/cucumber-reports/register-report.html
CartTestRunner	cart.feature	@cart	target/cucumber-reports/cart-report.html

All runners share the same glue packages: com.precision.ui.stepdefs and com.precision.ui.hooks. Parallel execution is disabled (parallel = false) to preserve browser session integrity per scenario.

6. Data-Driven Testing with Excel

All test input data is stored in testdata/testdata.xlsx on the classpath. The ExcelUtils class (from the common module) reads data by sheet name and test case ID, returning a Map<String, String> that maps column headers to cell values.

Sheet Name	Test Case ID	Columns Used
RegisterData	TC1_RegisterUser	name, title, password, day, month, year, firstName, lastName, address, country, state, city, zipcode, mobile
LoginData	TC2_ValidLogin	email, password
LoginData	TC3_InvalidLogin	email, password
CartData	TC4_AddToCart	productIndex, expectedProductName, expectedQuantity
CartData	TC5_RemoveFromCart	productIndex (reused from TC4 context)

Unique Email Generation for Registration

Since AutomationExercise rejects duplicate email addresses, RegisterSteps generates a unique email per run using System.currentTimeMillis():

```
String uniqueEmail = "testuser_" + System.currentTimeMillis() + "@gmail.com";
```

This ensures every registration test run succeeds without cleanup between runs.

7. Execution Instructions

7.1 Prerequisites

- Java 17 or above installed
- Maven 3.8+ installed
- Chrome / Firefox / Edge browser installed
- config.properties configured with base.url, browser, explicit.wait
- testdata.xlsx present at src/test/resources/testdata/testdata.xlsx

7.2 Run Commands

```
# Run all UI tests
mvn test -pl ui-module

# Run only login tests
mvn test -pl ui-module -Dcucumber.filter.tags="@login"

# Run only register tests
mvn test -pl ui-module -Dcucumber.filter.tags="@register"

# Run only cart tests
mvn test -pl ui-module -Dcucumber.filter.tags="@cart"
```

7.3 Report Locations

```
target/extent-reports/AutomationReport.html      (ExtentReports - Dark Theme)
target/cucumber-reports/login-report.html        (Cucumber HTML)
target/cucumber-reports/register-report.html     (Cucumber HTML)
target/cucumber-reports/cart-report.html         (Cucumber HTML)
target/screenshots/                              (Failure Screenshots - PNG)
```

7.4 config.properties Keys Used by ui-module

Key	Purpose
base.url	Root URL of the application under test (https://automationexercise.com)
browser	Browser to launch — chrome, firefox, or edge
explicit.wait	Timeout in seconds for WebDriverWait — used in all WaitUtils methods

8. Challenges Faced & Solutions

Challenge	Solution
Google Ads overlaying UI elements — buttons blocked by ad iframes, causing ElementClickInterceptedException on login, registration, and add-to-cart actions	Implemented dismissAdsIfPresent() in BasePage using JavaScript to set display:none on all known ad selectors. Called before every critical interaction in page classes and step definitions.
Login button click failing intermittently despite element being visible — real-world browser rendering and overlay timing issues	Implemented a 3-level fallback click chain in loginAs(): standard click → JS click → Actions click. Each fallback is tried only if the previous one throws an exception, with debug output at each stage.
Registration email uniqueness — reusing the same email address in repeated test runs causes 'Email already registered' failures	Generated a timestamp-based unique email (testuser_<millis>@gmail.com) in RegisterSteps before each test run. The generated email is stored back into the data map for reuse across steps.
Cart item removal — WebDriverWait failing after clicking delete because the element reference becomes stale before invisibility check completes	Used WaitUtils.waitForInvisibility(cartItems.get(index)) immediately after click. The invisibility wait uses the already-resolved element reference which becomes detached from DOM on removal.
Create Account button scroll interception — button at bottom of long registration form scrolled behind sticky elements or ad banners	Combined dismissAdsIfPresent() + scrollToElement(createAccountBtn) + Thread.sleep(1000) + jsClick(createAccountBtn) sequence to ensure reliable interaction regardless of page scroll state.

9. Conclusion & Future Enhancements

9.1 Summary

The ui-module successfully automates all 5 critical user journeys of the AutomationExercise e-commerce application. The Page Object Model design pattern ensures clean separation between page interactions and test logic, making the framework highly maintainable and extensible.

- 5 test scenarios covering registration, valid login, invalid login, add to cart, and remove from cart
- 6 page classes following strict POM — BasePage as the reusable foundation
- Data-driven approach via Apache POI with zero hardcoded test values
- Multi-browser support — Chrome, Firefox, and Edge via DriverManager
- Failure screenshots automatically captured and embedded in ExtentReports
- 3 independent TestNG runners for targeted execution by feature

9.2 Future Enhancements

4. Enable parallel browser execution using TestNG parallel='methods' and ThreadLocal WebDriver
5. Integrate with Jenkins / GitHub Actions CI/CD pipeline for automated regression on every commit
6. Add Selenium Grid or BrowserStack for cross-browser and cross-OS cloud execution
7. Implement visual regression testing using AShot or Applitools Eyes for pixel-level UI comparison
8. Add explicit JSON / XML data providers as an alternative to Excel for parameterized scenarios
9. Expand page coverage to include checkout flow, wishlist, and user account management pages

Appendix — Dependencies (pom.xml)

Dependency	Version	Purpose
org.seleniumhq.selenium : selenium-java	4.x	Browser automation — WebDriver, PageFactory, Actions
io.cucumber : cucumber-java	7.x	BDD step definitions and Gherkin annotations
io.cucumber : cucumber-testng	7.x	TestNG integration for Cucumber runner
org.testng : testng	7.x	Test runner, assertions, DataProvider
org.apache.poi : poi-ooxml	5.x	Excel .xlsx file reading for test data
com.aventstack : extentreports	5.x	HTML test reporting with Spark dark theme
org.apache.logging.log4j : log4j-core	2.x	Structured logging to console and file
commons-io : commons-io	2.x	FileUtils.copyFile() used in ScreenshotUtils
com.precision : common (module ref)	1.0	DriverManager, ConfigReader, WaitUtils, ExtentManager

— End of Document —